

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	<i>PRUEBAS DE SOFTWARE</i>				
TÍTULO DE LA PRÁCTICA:	<i>Pruebas Unitarias con Pytest</i>				
NÚMERO DE PRÁCTICA:	<i>03</i>	AÑO LECTIVO:	<i>2026</i>	NRO. SEMESTRE:	<i>VII</i>
FECHA DE PRESENTACIÓN	<i>06/05/2026</i>	HORA DE PRESENTACIÓN	<i>23:59</i>		
INTEGRANTE (s): Quispe Flores Marco Ramiro Quispe Madariaga Jeferson Jofre Ramirez Ccahuana Max Edu Valdiviezo Tovar Alexander				NOTA:	
DOCENTE(s): Prof. Robert Arisaca					

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS Para cada uno de los siguientes ejercicios: a) Cree el programa en Python (módulo bajo prueba) b) Diseñe los casos de prueba necesarios en una tabla (Id, Descripción, Entrada, Resultado Esperado) c) Implemente las pruebas unitarias con Pytest aplicando el patrón AAA d) Valide que su programa pasa todos los casos de prueba (captura de pantalla del resultado pytest -v) 1. Calculadora Básica Enunciado: Escriba un módulo calculadora.py que implemente las cuatro operaciones aritméticas básicas: suma, resta, multiplicación y división. Para la división, el programa debe manejar adecuadamente la división por cero lanzando una excepción de tipo ValueError con el mensaje “No se puede dividir entre cero”. Objetivo de prueba: Verificar que cada operación produce el resultado correcto para entradas enteras, decimales y negativas; y que la división por cero es manejada como excepción.

El informe debe contener el código fuente:

- calculadora.py
- test_calculadora.py

Resolución

Se utilizó la metodología de Test Driven Development (TDD) y el patrón Arrange-Act-Assert (AAA). En “test_calculadora.py” se prepararon datos de entrada (Arrange) para cada operación de suma, resta, multiplicación y división (Act) con un caso de excepción al hacer una división entre 0, esperando el resultado correspondiente (Assert).

Python

```
import pytest
```

```
@pytest.mark.parametrize("a, b, esperado", [
    (5, 3, 8),
    (-4, 4, 0),
    (0, 0, 0),
    (-1, -1, -2),
    (2.5, 3.5, 6.0),
])
```

```
def test_suma(a, b, esperado):
    resultado = suma(a, b)
    assert resultado == esperado
```

```
@pytest.mark.parametrize("a, b, esperado", [
    (10, 3, 7),
    (3, 10, -7),
    (7, -3, 10),
    (0, 0, 0),
    (-1, -1, 0),
])
```

```
def test_resta(a, b, esperado):
    resultado = resta(a, b)
    assert resultado == esperado
```

```
@pytest.mark.parametrize("a, b, esperado", [
    (2, 3, 6),
    (7, -3, -21),
    (5, 0, 0),
    (-1, -1, 1),
    (2.5, 4, 10.0),
])
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

```
def test_multiplicacion(a, b, esperado):
    resultado = multiplicacion(a, b)
    assert resultado == esperado

@pytest.mark.parametrize("a, b, esperado", [
    (10, 2, 5.0),
    (10, -2, -5.0),
    (0, 1, 0.0),
    (-4, -2, 2.0),
    (7, 2, 3.5),
])
def test_division(a, b, esperado):
    resultado = division(a, b)
    assert resultado == esperado

def test_division_por_cero():
    with pytest.raises(ValueError):
        division(5, 0)
```

Como es esperado las pruebas fallaron, debido a que, aún no se encuentran implementadas las funciones necesarias para las operaciones (fase RED).

Ejecución de pruebas

```
FAILED test_calculadora.py::test_division[10-2-5.0] - NameError: name 'division' is not defined
FAILED test_calculadora.py::test_division[10--2--5.0] - NameError: name 'division' is not defined
FAILED test_calculadora.py::test_division[0-1-0.0] - NameError: name 'division' is not defined
FAILED test_calculadora.py::test_division[-4--2-2.0] - NameError: name 'division' is not defined
FAILED test_calculadora.py::test_division[7-2-3.5] - NameError: name 'division' is not defined
FAILED test_calculadora.py::test_division_por_cero - NameError: name 'division' is not defined
===== 21 failed in 0.73s =====
PS C:\Users\OFICINA\Documents\LabPS\Lab03>
```

Se crearon las funciones mínimas necesarias en “calculadora.py” con el objetivo de pasar las pruebas (fase GREEN).

```
Python
def suma(a, b):
    return a + b

def resta(a, b):
    return a - b

def multiplicacion(a, b):
```

```
return a * b
```

```
def division(a, b):  
    return a / b
```

Ejecución de pruebas

```
PS C:\Users\OFICINA\Documents\LabPS\Lab03> py -m pytest  
===== test session starts =====  
platform win32 -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0  
rootdir: C:\Users\OFICINA\Documents\LabPS\Lab03  
collected 21 items  
  
test_calculadora.py .....F [100%]  
  
===== FAILURES =====  
_____ test_division_por_cero _____  
  
    def test_division_por_cero():  
        with pytest.raises(ValueError):  
>         division(5, 0)  
  
test_calculadora.py:50:  
-----  
a = 5, b = 0  
  
    def division(a, b):  
>     return a / b  
        ^^^^^  
E       ZeroDivisionError: division by zero  
  
calculadora.py:11: ZeroDivisionError  
===== short test summary info =====  
FAILED test_calculadora.py::test_division_por_cero - ZeroDivisionError: division by zero  
===== 1 failed, 20 passed in 0.28s =====  
PS C:\Users\OFICINA\Documents\LabPS\Lab03>
```

Se observa que la gran mayoría de pruebas pasaron, sin embargo, falla la división entre 0, esto se debe a que no se maneja aún la excepción ValueError en la función “def division”, se implementa el manejo de excepciones para dicho caso y robustez en caso de ingresar caracteres o cadenas de texto (fase REFACTOR).

Además se agregaron más pruebas para verificar que el manejo de excepciones funciona.

calculadora.py

Python

```
def suma(a, b):
    validar_argumentos(a, b)
    return a + b

def resta(a, b):
    validar_argumentos(a, b)
    return a - b

def multiplicacion(a, b):
    validar_argumentos(a, b)
    return a * b

def division(a, b):
    validar_argumentos(a, b)
    if b == 0:
        raise ValueError("No se puede dividir entre cero")
    return a / b

def validar_argumentos(a, b):
    if not (isinstance(a, (int, float)) and isinstance(b, (int, float))):
        raise TypeError("Argumentos inválidos, solo ingresar números")
```

test_calculadora.py

Python

```
import pytest
from calculadora import suma, resta, multiplicacion, division

"""Pruebas unitarias para calculadora con patron Arrange-Act-Assert"""

@pytest.mark.parametrize("a, b, esperado", [
    (5, 3, 8),
    (-4, 4, 0),
    (0, 0, 0),
    (-1, -1, -2),
    (2.5, 3.5, 6.0),
])

# Test de suma
def test_suma(a, b, esperado):
    # Arrange, datos en parametrize
    # Act
```

```
    resultado = suma(a, b)
    # Assert
    assert resultado == esperado

@pytest.mark.parametrize("a, b, esperado", [
    (10, 3, 7),
    (3, 10, -7),
    (7, -3, 10),
    (0, 0, 0),
    (-1, -1, 0),
])
# Test de resta
def test_resta(a, b, esperado):
    # Arrange, datos en parametrize
    # Act
    resultado = resta(a, b)
    # Assert
    assert resultado == esperado

@pytest.mark.parametrize("a, b, esperado", [
    (2, 3, 6),
    (7, -3, -21),
    (5, 0, 0),
    (-1, -1, 1),
    (2.5, 4, 10.0),
])
# Test de multiplicación
def test_multiplicacion(a, b, esperado):
    # Arrange, datos en parametrize
    # Act
    resultado = multiplicacion(a, b)
    # Assert
    assert resultado == esperado

@pytest.mark.parametrize("a, b, esperado", [
    (10, 2, 5.0),
    (10, -2, -5.0),
    (0, 1, 0.0),
    (-4, -2, 2.0),
    (7, 2, 3.5),
])
# Test de división
def test_division(a, b, esperado):
```

```
# Arrange, datos en parametrize
# Act
resultado = division(a, b)
# Assert
assert resultado == esperado

# Test de división por cero
def test_division_por_cero():
    with pytest.raises(ValueError):
        division(5, 0)

@pytest.mark.parametrize("operacion, a, b", [
    (suma, "a", 3),
    (resta, 5, "cinco"),
    (multiplicacion, "?", 4),
    (division, None, 2),
])

# Test de argumentos inválidos
def test_argumentos_invalidos(operacion, a, b):
    # Arrange, datos en parametrize
    # Act y Assert
    with pytest.raises(TypeError):
        operacion(a, b)
```

Ejecución de pruebas

```
PS C:\Users\OFICINA\Documents\LabPS\Lab03> py -m pytest -v
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\OFICINA\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\OFICINA\Documents\LabPS\Lab03
collected 25 items

test_calculadora.py::test_suma[5-3-8] PASSED [ 4%]
test_calculadora.py::test_suma[-4-4-0] PASSED [ 8%]
test_calculadora.py::test_suma[0-0-0] PASSED [ 12%]
test_calculadora.py::test_suma[-1--1--2] PASSED [ 16%]
test_calculadora.py::test_suma[2.5-3.5-6.0] PASSED [ 20%]
test_calculadora.py::test_resta[10-3-7] PASSED [ 24%]
test_calculadora.py::test_resta[3-10--7] PASSED [ 28%]
test_calculadora.py::test_resta[7--3-10] PASSED [ 32%]
test_calculadora.py::test_resta[0-0-0] PASSED [ 36%]
test_calculadora.py::test_resta[-1--1-0] PASSED [ 40%]
test_calculadora.py::test_multiplicacion[2-3-6] PASSED [ 44%]
test_calculadora.py::test_multiplicacion[7--3--21] PASSED [ 48%]
test_calculadora.py::test_multiplicacion[5-0-0] PASSED [ 52%]
test_calculadora.py::test_multiplicacion[-1--1-1] PASSED [ 56%]
test_calculadora.py::test_multiplicacion[2.5-4-10.0] PASSED [ 60%]
test_calculadora.py::test_division[10-2-5.0] PASSED [ 64%]
test_calculadora.py::test_division[10--2--5.0] PASSED [ 68%]
test_calculadora.py::test_division[0-1-0.0] PASSED [ 72%]
test_calculadora.py::test_division[-4--2-2.0] PASSED [ 76%]
test_calculadora.py::test_division[7-2-3.5] PASSED [ 80%]
test_calculadora.py::test_division_por_cero PASSED [ 84%]
test_calculadora.py::test_argumentos_invalidos[suma-a-3] PASSED [ 88%]
test_calculadora.py::test_argumentos_invalidos[resta-5-cinco] PASSED [ 92%]
test_calculadora.py::test_argumentos_invalidos[multiplicacion-?-4] PASSED [ 96%]
test_calculadora.py::test_argumentos_invalidos[division-None-2] PASSED [100%]

===== 25 passed in 0.19s =====
```

Guía de casos de prueba mínimos requeridos:

Id.	Descripción	Entrada	Resultado Esperado	Resultado Real
TC-01	Suma de dos enteros positivos	(5, 3)	8	PASS
TC-02	Suma con número negativo	(-4, 4)	0	PASS
TC-03	Resta con resultado positivo	(10, 3)	7	PASS
TC-04	Resta con resultado negativo	(3, 10)	-7	PASS
TC-05	Multiplicación por cero	(5, 0)	0	PASS

TC-06	Multiplicación de números decimales	(2.5, 4)	10.0	PASS
TC-07	División exacta	(10, 2)	5.0	PASS
TC-08	División por cero lanza excepción	(5, 0)	ValueError (Error: No se puede dividir entre cero)	PASS
TC-09	División con resultado decimal	(7, 2)	3.5	PASS
TC-10	Suma con argumento inválido lanza excepción	("a", 3)	TypeError (Error: Argumentos inválidos, solo ingresar números)	PASS
TC-11	División con argumento inválido lanza excepción	(10, None)	TypeError (Error: Argumentos inválidos, solo ingresar números)	PASS

2. Validador de Contraseñas

Enunciado: Implemente un módulo validador.py con la función validar_contrasena(contrasena: str) -> dict que evalúe la fortaleza de una contraseña según las siguientes reglas:

- Longitud mínima de 8 caracteres.
- Contiene al menos una letra mayúscula.
- Contiene al menos una letra minúscula.
- Contiene al menos un dígito numérico.
- Contiene al menos un carácter especial de entre: ! @ # \$ % ^ & *

La función debe retornar un diccionario con la clave "valida" (True/False) y la clave "errores" (lista de strings con los criterios no cumplidos). Si la contraseña cumple todos los criterios, "errores" debe ser una lista vacía.

Objetivo de prueba: Comprobar que el validador detecta correctamente cada regla incumplida de forma individual y en combinación; y que acepta contraseñas que cumplen todos los criterios.

El informe debe contener el código fuente:

- validador.py
- test_validador.py

validador.py

(Este código fue desarrollado con ayuda del [artículo de GeeksforGeeks sobre expresiones generadoras](#))

Python

```
def validar_contraseña(contraseña: str) -> dict:

    errores = []

    if len(contraseña) < 8:
        errores.append("longitud")

    if not any(c.isupper() for c in contraseña):
        errores.append("mayuscula")

    if not any(c.islower() for c in contraseña):
        errores.append("minuscula")

    if not any(c.isdigit() for c in contraseña):
        errores.append("digito")

    especiales = "!@#$%^&*"
    if not any(c in especiales for c in contraseña):
        errores.append("especial")

    return {
        "valida": len(errores) == 0,
        "errores": errores
    }
```

Esta implementación analiza cada criterio por separado y agrega el nombre del error correspondiente en la lista errores. Finalmente, se retorna el diccionario con el resultado de la validación.

test_validador.py

Python

```
import pytest
from validador import validar_contraseña

def test_contraseña_valida():
    resultado = validar_contraseña("Segura#1")
    assert resultado["valida"] is True
    assert resultado["errores"] == []

def test_contraseña_muy_corta():
    resultado = validar_contraseña("Ab1!")
```

```
assert resultado["valida"] is False
assert "longitud" in resultado["errores"]

def test_sin_mayuscula():
    resultado = validar_contrasena("segura#1")
    assert resultado["valida"] is False
    assert "mayuscula" in resultado["errores"]

def test_sin_minuscula():
    resultado = validar_contrasena("SEGURA#1")
    assert resultado["valida"] is False
    assert "minuscula" in resultado["errores"]

def test_sin_digito():
    resultado = validar_contrasena("Segura##")
    assert resultado["valida"] is False
    assert "digito" in resultado["errores"]

def test_sin_caracter_especial():
    resultado = validar_contrasena("Segura12")
    assert resultado["valida"] is False
    assert "especial" in resultado["errores"]

def test_contrasena_vacia():
    resultado = validar_contrasena("")
    assert resultado["valida"] is False
    assert len(resultado["errores"]) >= 4

@pytest.mark.parametrize("contrasena", [
    "abc",
    "password",
    "ABCDEFGH",
    "12345678",
])

def test_contrasenas_debiles(contrasena):
    resultado = validar_contrasena(contrasena)
    assert resultado["valida"] is False

def test_contrasena_8_caracteres_valida():
    resultado = validar_contrasena("aB1!cDe2")
    assert resultado["valida"] is True
    assert resultado["errores"] == []
```

Ejecución:

```
PS D:\PruSofLab\03> python -m pytest -v
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\Hydro\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
rootdir: D:\PruSofLab\03
collected 12 items

test_validador.py::test_contrasena_valida PASSED [ 8%]
test_validador.py::test_contrasena_muy_corta PASSED [ 16%]
test_validador.py::test_sin_mayuscula PASSED [ 25%]
test_validador.py::test_sin_minuscula PASSED [ 33%]
test_validador.py::test_sin_digito PASSED [ 41%]
test_validador.py::test_sin_caracter_especial PASSED [ 50%]
test_validador.py::test_contrasena_vacia PASSED [ 58%]
test_validador.py::test_contrasenas_debiles[abc] PASSED [ 66%]
test_validador.py::test_contrasenas_debiles[password] PASSED [ 75%]
test_validador.py::test_contrasenas_debiles[ABCDEFGH] PASSED [ 83%]
test_validador.py::test_contrasenas_debiles[12345678] PASSED [ 91%]
test_validador.py::test_contrasena_8_caracteres_valida PASSED [100%]

===== 12 passed in 0.02s =====
PS D:\PruSofLab\03>
```

Casos de Prueba:

En esta tabla se documentan las pruebas predefinidas en la guía del docente, ya comprobadas:

ID	Descripción	Entrada	Resultado Esperado	Resultado Real
TC-01	Contraseña completamente válida	"Segura#1"	valida: True errores: []	PASS
TC-02	Contraseña muy corta (<8 caracteres)	"Ab1!"	valida: False error longitud	PASS
TC-03	Sin letra mayúscula	"segura#1"	valida: False error mayúscula	PASS
TC-04	Sin letra minúscula	"SEGURA#1"	valida: False error minúscula	PASS
TC-05	Sin dígito numérico	"Segura##"	valida: False error dígito	PASS
TC-06	Sin carácter especial	"Segura12"	valida: False error especial	PASS
TC-07	Contraseña vacía (múltiples	""	valida: False	PASS

	errores)		5 errores	
TC-08	Exactamente 8 caracteres válidos	"aB1!cDe2"	valida: True errores: []	PASS
TC-09	Parametrizado: múltiples contraseñas débiles	ver código	valida: False en todos	PASS

3. Simulador de Cajero Automático con Pruebas Unitarias

Enunciado: Retome el ejercicio del Mini Cajero Automático de la Guía LAB_01. Esta vez, refactorice la lógica de negocio en una clase Atm (archivo atm.py) con los métodos consultar_saldo(), depositar(monto) y retirar(monto). Aplique el mismo esquema de excepciones de la clase BankAccount del Ejercicio 3.2 de esta guía. Luego, escriba un archivo test_atm.py que valide toda la lógica usando Pytest, fixtures y parametrize. Objetivo de prueba: Verificar que el flujo completo del cajero es correcto: consulta de saldo, depósitos válidos e inválidos, retiros válidos e inválidos (incluyendo casos límite como retirar exactamente el saldo disponible), y el manejo correcto de todas las excepciones. El informe debe contener el código fuente:

- atm.py (clase Atm con lógica de negocio, sin llamadas a input())
- test_atm.py (suite completa de pruebas Pytest con fixture y parametrize)

Guía de casos de prueba mínimos requeridos (diseñe al menos 12 casos):

Para desarrollar el ejercicio se tomó como base el código implementado en el "Laboratorio 01", el cual simulaba las operaciones básicas de un cajero automático mediante funciones y un menú interactivo utilizando input() y print().

Posteriormente, siguiendo la guía y el ejemplo desarrollado en el Ejercicio 3.2 del laboratorio actual ("BankAccount"), se realizó una modificación del programa para adaptar la lógica para que validara mediante pruebas unitarias con Pytest.

- Se transformó el programa basado en funciones en una clase llamada Atm
- Se eliminó toda interacción directa con el usuario (input() y print()) Para ejecutar las pruebas unitarias
- Se encapsulo el atributo saldo
- Se agrego excepciones "SaldoInsuficienteError" y "MontoInvalidoError"

atm.py

Python

atm.py

```
class SaldoInsuficienteError(Exception):
    """Se lanza cuando se intenta retirar más del saldo disponible."""
    pass

class MontoInvalidoError(Exception):
    """Se lanza cuando el monto es cero o negativo."""
    pass

class Atm:
    def __init__(self, saldo_inicial=1000.0):
        if saldo_inicial < 0:
            raise MontoInvalidoError("Saldo inicial no puede ser negativo")
        self._saldo = saldo_inicial

    def consultar_saldo(self):
        return self._saldo

    def depositar(self, monto):
        if monto <= 0:
            raise MontoInvalidoError("El monto debe ser mayor a cero")
        self._saldo += monto

    def retirar(self, monto):
        if monto <= 0:
            raise MontoInvalidoError("El monto debe ser mayor a cero")

        if monto > self._saldo:
            raise SaldoInsuficienteError("Saldo insuficiente")

        self._saldo -= monto
```

test_atm.py

Python

import pytest

```
from atm import Atm, SaldoInsuficienteError, MontoInvalidoError
```

```
@pytest.fixture
```

```
def atm():  
    """Fixture: retorna un Atm con S/.1000 de saldo inicial."""  
    return Atm(1000.0)
```

```
#----- Pruebas: saldo, depósito y retiro válidos -----
```

```
def test_saldo_inicial(atm):  
    assert atm.consultar_saldo() == 1000.0
```

```
def test_deposito_valido(atm):  
    atm.depositar(500)  
    assert atm.consultar_saldo() == 1500.0
```

```
def test_retiro_valido(atm):  
    atm.retirar(300)  
    assert atm.consultar_saldo() == 700.0
```

```
def test_retiro_total(atm):  
    atm.retirar(1000)  
    assert atm.consultar_saldo() == 0.0
```

```
#----- Pruebas: excepciones -----
```

```
def test_retiro_excede(atm):  
    with pytest.raises(SaldoInsuficienteError):  
        atm.retirar(2000)
```

```
def test_deposito_negativo(atm):  
    with pytest.raises(MontoInvalidoError):  
        atm.depositar(-100)
```

```
def test_deposito_cero(atm):  
    with pytest.raises(MontoInvalidoError):  
        atm.depositar(0)
```

```
def test_retiro_negativo(atm):  
    with pytest.raises(MontoInvalidoError):  
        atm.retirar(-50)
```

```
def test_saldo_inicial_negativo():
    with pytest.raises(MontoInvalidoError):
        Atm(-500)

#----- Pruebas: parametrizadas -----
@pytest.mark.parametrize("depositos, saldo", [
    ([100, 200, 300], 1600.0)
])
def test_depositos_multiples(atm, depositos, saldo):
    for d in depositos:
        atm.depositar(d)
    assert atm.consultar_saldo() == saldo

@pytest.mark.parametrize("retiros, saldo", [
    ([100, 200, 300], 400.0)
])
def test_retiros_multiples(atm, retiros, saldo):
    for r in retiros:
        atm.retirar(r)
    assert atm.consultar_saldo() == saldo

def test_consultar_no_modifica(atm):
    atm.consultar_saldo()
    atm.consultar_saldo()
    assert atm.consultar_saldo() == 1000.0
```

Ejecución:

Durante la ejecución, Pytest detectó automáticamente todas las funciones cuyo nombre inicia con test_ y ejecutó cada prueba definida en test_atm.py.

```
PS C:\Users\jofre\OneDrive\Desktop\Lab\1.- Pruebas_de_software\lab03> python -m pytest test_atm.py -v
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0 -- C:\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\jofre\OneDrive\Desktop\Lab\1.- Pruebas_de_software\lab03
collected 12 items

test_atm.py::test_saldo_inicial PASSED [ 8%]
test_atm.py::test_deposito_valido PASSED [ 16%]
test_atm.py::test_retiro_valido PASSED [ 25%]
test_atm.py::test_retiro_total PASSED [ 33%]
test_atm.py::test_retiro_excede PASSED [ 41%]
test_atm.py::test_deposito_negativo PASSED [ 50%]
test_atm.py::test_deposito_cero PASSED [ 58%]
test_atm.py::test_retiro_negativo PASSED [ 66%]
test_atm.py::test_saldo_inicial_negativo PASSED [ 75%]
test_atm.py::test_depositos_multiples[depositos0-1600.0] PASSED [ 83%]
test_atm.py::test_retiros_multiples[retiros0-400.0] PASSED [ 91%]
test_atm.py::test_consultar_no_modifica PASSED [100%]

===== 12 passed in 0.05s =====
```


ID	Descripción	Entrada	Resultado Esperado	Resultado Real
TC-01	Saldo inicial correcto	saldo_inicial=1000	saldo == 1000.0	PASS
TC-02	Depósito válido	depositar(500)	saldo == 1500.0	PASS
TC-03	Retiro válido	retirar(300)	saldo == 700.0	PASS
TC-04	Retiro exacto al saldo disponible	retirar(1000)	saldo == 0.0	PASS
TC-05	Retiro mayor al saldo	retirar(1001)	SaldoInsuficienteError	PASS
TC-06	Depósito de monto negativo	depositar(-200)	MontoInvalidoError	PASS
TC-07	Depósito de monto cero	depositar(0)	MontoInvalidoError	PASS
TC-08	Retiro de monto negativo	retirar(-50)	MontoInvalidoError	PASS
TC-09	Saldo inicial negativo	saldo_inicial=-500	MontoInvalidoError	PASS
TC-10	Múltiples depósitos acumulados (parametrized)	[100, 200, 300]	saldo == 1600.0	PASS
TC-11	Múltiples retiros acumulados (parametrized)	[100, 200, 300]	saldo == 400.0	PASS
TC-12	Consulta de saldo no modifica el estado	consultar_saldo() x3	saldo inalterado	PASS

II. CUESTIONARIO

1. ¿Cuál es la diferencia fundamental entre una prueba unitaria y una prueba de integración? Proporcione un ejemplo concreto para cada tipo en el contexto de un sistema bancario.

La diferencia principal radica en el nivel de alcance de la verificación. Las pruebas unitarias se enfocan en validar componentes individuales del software —como funciones, métodos o clases— de manera aislada, generalmente sustituyendo sus dependencias por simulaciones o mocks para asegurar que el comportamiento interno sea correcto sin interferencias externas [1]. En cambio, las pruebas de integración evalúan el funcionamiento conjunto de múltiples módulos, comprobando que la interacción entre ellos (por ejemplo, entre servicios, bases de datos o APIs) sea coherente y produzca los resultados esperados [2].

En otras palabras, mientras las pruebas unitarias buscan garantizar que cada pieza del sistema funcione correctamente por sí sola, las pruebas de integración aseguran que dichas piezas colaboren adecuadamente dentro del sistema completo, permitiendo detectar fallos en la comunicación o en el flujo de datos entre componentes.

1. Prueba unitaria (aislada)

En este caso, se evalúa exclusivamente la lógica de débito de una cuenta bancaria. La prueba se centra únicamente en el correcto funcionamiento del método debitar, verificando que el saldo se actualice adecuadamente y que se manejen correctamente casos como fondos insuficientes. Todo esto se realiza de manera aislada, sin involucrar una base de datos ni la interacción con otros sistemas o componentes externos.

`class Cuenta:`

```
def __init__(self, saldo):  
    self.saldo = saldo
```

```
def debitar(self, monto):  
    if monto > self.saldo:  
        raise ValueError("Fondos insuficientes")  
    self.saldo -= monto
```

```
def test_debitar():  
    cuenta = Cuenta(500)  
    cuenta.debitar(100)  
    assert cuenta.saldo == 400
```

2. Prueba de integración (flujo completo)

En este tipo de prueba se evalúa la operación completa de una transferencia bancaria, considerando la interacción entre múltiples componentes del sistema. Esto incluye la lógica de negocio encargada de procesar la transferencia, la base de datos donde se consultan y actualizan los saldos, y el registro de la transacción realizada. El objetivo es verificar que todos estos elementos funcionen de manera conjunta y coherente, asegurando que el sistema completo opere correctamente como un todo.

```
def transferir(id_origen, id_destino, monto):
    cuenta_origen = db.obtener_cuenta(id_origen)
    cuenta_destino = db.obtener_cuenta(id_destino)
    cuenta_origen.debitar(monto)
    cuenta_destino.acreditar(monto)
    db.guardar(cuenta_origen)
    db.guardar(cuenta_destino)
    db.registrar_transaccion(id_origen, id_destino, monto)

def test_transferencia_integracion():
    db.inicializar_datos_prueba()
    transferir("A123", "B456", 200)
    saldo_origen = db.obtener_cuenta("A123").saldo
    saldo_destino = db.obtener_cuenta("B456").saldo
    assert saldo_origen == 800
    assert saldo_destino == 1200
    transaccion = db.obtener_ultima_transaccion()
    assert transaccion.monto == 200
```

2. Explique el principio F.I.R.S.T. de las pruebas unitarias (Fast, Independent, Repeatable, Selfvalidating, Timely). ¿Por qué el atributo "Independent" es especialmente importante al usar fixtures en Pytest?

El principio F.I.R.S.T. establece un conjunto de buenas prácticas que deben cumplir las pruebas unitarias para garantizar su calidad y utilidad en el desarrollo de software. En primer lugar, estas deben ser rápidas (Fast), permitiendo su ejecución frecuente; además, deben ser independientes (Independent), de modo que cada prueba funcione sin depender de otras. También deben ser repetibles (Repeatable), es decir, ofrecer los mismos resultados bajo las mismas condiciones, y auto-verificables (Self-validating), indicando claramente si han pasado o fallado sin necesidad de revisión manual. Finalmente, deben ser oportunas (Timely), lo que implica que se desarrollen en el momento adecuado, preferentemente junto con el código o incluso antes de este [3].

Dentro de estos principios, la independencia cobra especial relevancia al trabajar con fixtures en Pytest, ya que estos mecanismos se utilizan para preparar el entorno necesario para cada prueba. Si un fixture comparte o altera estados sin control —como datos en memoria o registros en una base de datos— puede generar dependencias implícitas entre pruebas. Esto ocasiona que el resultado de una prueba dependa del estado dejado por otra, rompiendo así el aislamiento esperado y afectando la confiabilidad del sistema de pruebas [4].

Por ello, garantizar la independencia implica diseñar fixtures que aislen completamente el contexto de cada prueba, asegurando que puedan ejecutarse en cualquier orden sin alterar los resultados. Pytest facilita este objetivo mediante el manejo de distintos alcances (scopes) y la reinicialización de datos cuando corresponde. De esta forma, se evitan efectos secundarios no deseados y se mantiene la consistencia en la ejecución de las pruebas, lo cual es fundamental para detectar errores de manera precisa y eficiente.

3. En el patrón AAA (Arrange–Act–Assert), ¿qué problemas puede ocasionar colocar múltiples assertions (assert) en una misma función de prueba? ¿Cuándo se considera aceptable hacerlo?

En el patrón AAA (Arrange–Act–Assert), incluir múltiples afirmaciones (asserts) dentro de una misma prueba puede generar problemas de claridad y mantenimiento. Cuando una prueba valida demasiadas condiciones a la vez, se vuelve más difícil identificar con precisión la causa de un fallo, ya que no queda claro qué verificación específica falló primero. Además, esto puede indicar que la prueba está evaluando múltiples comportamientos en lugar de centrarse en un único caso, lo que contradice el principio de pruebas simples y bien definidas [5].

Otro inconveniente es que las pruebas con múltiples asserts tienden a ser más frágiles y complejas de mantener. Si una de las condiciones cambia, puede ser necesario modificar toda la prueba, incluso si las demás verificaciones siguen siendo válidas. Esto también afecta la legibilidad, ya que el propósito de la prueba puede volverse ambiguo al intentar cubrir varios escenarios en una sola función, dificultando su comprensión para otros desarrolladores o para futuras revisiones del código [6].

Sin embargo, el uso de múltiples asserts puede considerarse aceptable cuando todas las verificaciones están relacionadas con un mismo comportamiento o resultado lógico. Por ejemplo, al validar el estado completo de un objeto después de una operación, donde varias propiedades deben comprobarse como parte de un único escenario coherente. En estos casos, los asserts no representan pruebas independientes, sino distintas facetas de una misma verificación, manteniendo así la coherencia con el patrón AAA [7].

4. Con respecto a Test-Driven Development (TDD) y sus tres fases (Red–Green–Refactor). ¿De qué manera las pruebas unitarias con Pytest facilitan la adopción de TDD? Incluya un ejemplo

sencillo en Python.

El Desarrollo Guiado por Pruebas (Test-Driven Development, TDD) es una metodología que establece escribir primero las pruebas antes de desarrollar el código funcional, siguiendo el ciclo Red–Green–Refactor.[8]

Por otro lado, las pruebas unitarias con Pytest facilitan la adopción de TDD debido a distintos factores:

- **Sintaxis simple:** Permite escribir pruebas de forma rápida usando assert, facilitando así iniciar la fase Red.
- **Ejecución automática:** Pytest detecta y ejecuta todas las pruebas fácilmente, permitiendo verificar rápidamente el paso de Red a Green.
- **Manejo de excepciones:** Haciendo uso de “pytest.raises()” se pueden probar errores esperados.
- **Parametrización:** permite probar múltiples casos sin duplicar código, haciendo más eficiente la validación.

Estas características reducen la complejidad de escribir pruebas, lo que incentiva su uso desde el inicio del desarrollo.[9]

Ejemplo: En el presente laboratorio se aplicó TDD con Pytest implementando una función de división como parte del módulo de calculadora.

Fase Red:

Primero se escribió una prueba para verificar la división y el manejo de errores:

Python

```
def test_division_por_cero():  
    with pytest.raises(ValueError):  
        division(5, 0)
```

La prueba fallaba debido a que la función aún no controlaba la división por cero.

Fase Green:

Se implementó la funcionalidad mínima:

Python

```
def division(a, b):  
    return a / b
```

Sin embargo, esta implementación si bien manejaba la mayoría de casos aún no manejaba adecuadamente el error de división por cero.

Fase Refactor:

Se mejoró el código manejando la excepción y agregando validaciones adicionales para la entrada correcta de argumentos:

Python

```
def division(a, b):  
    validar_argumentos(a, b)  
    if b == 0:  
        raise ValueError("No se puede dividir entre cero")  
    return a / b  
  
def validar_argumentos(a, b):  
    if not (isinstance(a, (int, float)) and isinstance(b, (int, float))):  
        raise TypeError("Argumentos inválidos, solo ingresar números")
```

El uso de TDD permitió desarrollar la funcionalidad de manera incremental, asegurando que los cambios estuvieran respaldados por pruebas. Esto facilita la detección de errores, limpieza y mantenimiento del código..

5. **¿Qué es la cobertura de código (code coverage) y cómo se mide con el plugin pytest-cov? Investigue las métricas statement coverage y branch coverage. ¿Una cobertura del 100% garantiza que el software no tiene errores? Justifique su respuesta.**

La cobertura de código (code coverage) es una métrica que permite evaluar qué proporción del código fuente ha sido ejecutada durante la ejecución de pruebas automatizadas. En el ecosistema de Python, esta se puede medir fácilmente mediante el plugin pytest-cov, el cual se integra con Pytest para generar reportes que indican qué partes del código han sido cubiertas por las pruebas. Este plugin muestra resultados en porcentaje y puede detallar líneas no ejecutadas, ayudando a identificar áreas del código que requieren mayor validación [10].

Entre las métricas más importantes se encuentran el statement coverage y el branch coverage. El statement coverage mide el porcentaje de instrucciones individuales del código que han sido ejecutadas al menos una vez durante las pruebas. Por otro lado, el branch coverage evalúa si todas las posibles rutas de decisión (como las ramas de estructuras condicionales if, else, switch) han sido recorridas. Esta última métrica es más estricta, ya que no solo verifica que una línea se ejecute, sino que se prueben todas sus posibles salidas lógicas [11].

Sin embargo, alcanzar un 100% de cobertura de código no garantiza la ausencia de errores. Esto se debe a que la cobertura únicamente mide qué tanto código se ejecuta, pero no asegura que las pruebas sean correctas o que validen adecuadamente todos los escenarios posibles. Es posible tener pruebas que ejecuten todo el código pero no detecten fallos lógicos, casos límite o errores en los requisitos. Por ello, la cobertura debe considerarse como una guía para mejorar la calidad de las pruebas, pero no como una garantía absoluta de software libre de errores [12].

Presentamos un ejemplo básico

```
def dividir(a, b):  
    if b == 0:  
        raise ValueError("No se puede dividir entre cero")  
    return a / b
```

Las pruebas serían

```
import pytest  
from calculadora import dividir  
  
def test_dividir_normal():  
    assert dividir(10, 2) == 5  
  
def test_dividir_cero():  
    with pytest.raises(ValueError):  
        dividir(10, 0)
```

Ejecutamos con cobertura

```
E:\2026\PS\lab03>py -3.11 -m pytest --cov=calculadora  
===== test session starts =====  
platform win32 -- Python 3.11.9, pytest-9.0.3, pluggy-1.6.0  
rootdir: E:\2026\PS\lab03  
plugins: cov-7.1.0  
collected 2 items  
  
test_calculadora.py .. [100%]  
  
===== tests coverage =====  
----- coverage: platform win32, python 3.11.9-final-0 -----  
  
Name           Stmts  Miss  Cover  
-----  
calculadora.py      4      0   100%  
TOTAL              4      0   100%  
  
===== 2 passed in 0.13s =====
```

III. CONCLUSIONES

- La aplicación del enfoque Test-Driven Development (TDD) permitió desarrollar las funcionalidades de manera incremental y controlada, siguiendo las fases Red–Green–Refactor. Esto facilitó la detección temprana de errores y garantizó que cada componente implementado cumpliera con los requisitos definidos desde el inicio.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 24

- El uso de Pytest demostró ser una herramienta eficiente para la automatización de pruebas, gracias a su sintaxis simple, soporte de parametrización y manejo de excepciones, lo que permitió cubrir múltiples escenarios de prueba de forma clara y organizada.
- La implementación del patrón Arrange–Act–Assert (AAA) contribuyó a estructurar adecuadamente las pruebas unitarias, mejorando la legibilidad, mantenibilidad y comprensión del código de pruebas, lo cual es fundamental en proyectos de mayor escala.
- El desarrollo de módulos como la calculadora y el validador de contraseñas evidenció la importancia de considerar tanto casos válidos como inválidos, incluyendo el manejo de excepciones, lo que incrementa la robustez y confiabilidad del software.
- Finalmente, el uso de métricas como la cobertura de código permitió evaluar el nivel de pruebas realizadas; sin embargo, se concluye que alcanzar un alto porcentaje de cobertura no garantiza la ausencia de errores, sino que debe complementarse con pruebas bien diseñadas y escenarios representativos del comportamiento real del sistema .

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

- [1] Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall, pp. 131–144.
- [2] Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner’s Approach (9th ed.). McGraw-Hill, pp. 733–750.
- [3] Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall, pp. 122–130.
- [4] Okken, B. (2017). Python Testing with pytest: Simple, Rapid, Effective, and Scalable. Pragmatic Bookshelf, pp. 45–70.
- [5] Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall, pp. 9–12.
- [6] Meszaros, G. (2007). xUnit test patterns: Refactoring test code. Addison-Wesley. Recuperado de: <https://github.com/Dannyb48/SoftwareTestingBooks/blob/master/xUnit%20Test%20Patterns%3A%20Refactoring%20Test%20Code%20-%20by%20Gerard%20Meszaros%20-%202007.pdf>
- [7] Fowler, M. (2018). Refactoring: Improving the design of existing code (2nd ed.). Addison-Wesley. Recuperado de: <https://martinfowler.com/books/refactoring.html>
- [8] Beck, K. (2003). Test-driven development: By example. Addison-Wesley, pp. 1–20. Recuperado de: <https://github.com/zakirullin/cs-books/blob/master/Test%20Driven%20Development%20By%20Example%20-%20Kent%20Beck.pdf>
- [9] Okken, B. (2017). Python testing with pytest: Simple, rapid, effective, and scalable. Pragmatic Bookshelf, pp. 1–50. Recuperado de: <https://pragprog.com/titles/bopytest/python-testing-with-pytest/>
- [10] Okken, B. (2017). Python testing with pytest: Simple, rapid, effective, and scalable. Pragmatic Bookshelf, pp. 85–110. Recuperado de: <https://pragprog.com/titles/bopytest/python-testing-with-pytest/>
- [11] Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner’s approach (9th ed.). McGraw-Hill, pp. 753–760.
- [12] ISTQB. (2018). Foundation level syllabus, pp. 43–47. Recuperado de: <https://www.istqb.org/resources/syllabi.html>